SIMATS ENGINEERING
ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION COURSE CODE:ITA0515 Slot

COURSE OUTCOME COVERED
CO3: ***Analyze and extract image features using detection and matching techniques for effective image representation and comparison.*** **(BTL 4)**
Assessment Tool Weightage: 40%

QUESTIONS:3 Total Marks 30 Duration :60 Minutes Annexure A
Experiments

Code of Conduct:

I _Harshavardhan. RV (192472163)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary

action. Signature of the student: Harshavardhan

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	*Marks(100)*
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	

Documentation	**10**	**Clear, well-structured documentation with diagrams, code, and results**	**Organized documentation with necessary details**	**Basic documentation with missing details**	**Poor or incomplete documentation**	

1. Harris Corner Detection Analysis

Perform Harris corner detection on images containing varying textures and patterns. Explain the concept of corner detection and its importance in computer vision tasks. Use appropriate parameters and tools to execute the experiment, document the detected corner points clearly, and analyze the distribution, accuracy, and significance of the detected corners.

Aim

To perform **Harris Corner Detection** on images containing different textures and patterns, detect corner points accurately, and analyze their distribution and significance in computer vision applications.

Algorithm

1. Read the input image.
2. Convert the image into grayscale.
3. Convert the grayscale image into float format (float32).
4. Apply the Harris Corner Detection algorithm using:
    - Block Size
    - Sobel Kernel Size
    - Harris Detector Constant (k)
5. Dilate the detected corner response to enhance corner visibility.
6. Apply a threshold to identify strong corner points.
7. Mark the detected corners on the original image.
8. Display the original image and the detected corners.

9. Analyze the number and distribution of detected corners.

Code:

```
import cv2

import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("image.jpg")

gray = cv2.cvtColor(image,

cv2.COLOR_BGR2GRAY) gray = np.float32(gray)

dst = cv2.cornerHarris(gray, 2, 3, 0.04)

dst = cv2.dilate(dst, None)

image[dst > 0.01 * dst.max()] = [0, 0, 255]

plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))

plt.title("Harris Corner Detection")

plt.axis("off")

plt.show()
```

Output:

Result:

The program is executed successfully.

2. Effect of Noise on Corner Detection

Design an experiment to study the effect of noise on Harris corner detection performance. Explain the purpose of introducing noise and its influence on image features. Add noise systematically, perform corner detection using suitable tools, document observations for original and noisy images, and analyze the impact of noise along with possible improvement techniques.

Aim

To study the effect of noise on Harris Corner Detection by comparing corner detection results on original and noisy images and analyzing the impact of noise on corner detection accuracy.

Algorithm

1. Read the input image.
2. Convert the image into grayscale.
3. Add Gaussian noise to the image.
4. Convert both original and noisy images to float32.
5. Apply Harris Corner Detection on both images.
6. Dilate the corner response.
7. Apply a threshold to detect strong corners.
8. Mark the detected corners.
9. Display the original and noisy images with detected corners.
10. Compare and analyze the results.

Code :

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

image = cv2.imread("/content/images.png")
```

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
noise = np.random.normal(0, 25,

gray.shape).astype(np.uint8) noisy = cv2.add(gray, noise)

gray1 = np.float32(gray)

gray2 = np.float32(noisy)

dst1 = cv2.cornerHarris(gray1, 2, 3, 0.04)

dst2 = cv2.cornerHarris(gray2, 2, 3, 0.04)

dst1 = cv2.dilate(dst1, None)

dst2 = cv2.dilate(dst2, None)

img1 = image.copy()

img2 = cv2.cvtColor(noisy,

cv2.COLOR_GRAY2BGR) img1[dst1 > 0.01 *

dst1.max()] = [0, 0, 255]

img2[dst2 > 0.01 * dst2.max()] = [0, 0, 255]

plt.figure(figsize=(10,5))

plt.subplot(1,2,1)

plt.imshow(cv2.cvtColor(img1, cv2.COLOR_BGR2RGB))

plt.title("Original Image")

plt.axis("off")

plt.subplot(1,2,2)

plt.imshow(cv2.cvtColor(img2, cv2.COLOR_BGR2RGB))

plt.title("Noisy Image")

plt.axis("off")

plt.show()
```

Output:

Result: The program is executed successfully.

3. Hough Transform for Shape Detection

Conduct an experiment using the Hough Transform to detect lines or circles in an image. Explain the principles and significance of the Hough Transform in shape detection. Execute the detection process using appropriate tools, record the detected shapes and their parameters, and analyze the detection accuracy under different image conditions.

Aim

To detect lines or circles in an image using the Hough Transform and analyze the detection accuracy under different image conditions.

Algorithm

1. Read the input image.
2. Convert the image into grayscale.
3. Apply Gaussian Blur to reduce noise.
4. Detect edges using the Canny Edge Detector.
5. Apply the Hough Transform to detect lines or circles.
6. Draw the detected shapes on the original image.
7. Display the output image.
8. Analyze the detected shapes and their parameters.

Code

```
import cv2

import numpy as np

import matplotlib.pyplot as plt
```

```
image = cv2.imread("/content/images.png")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

blur = cv2.GaussianBlur(gray, (5,5), 0)

edges = cv2.Canny(blur, 50, 150)
lines = cv2.HoughLinesP(edges, 1, np.pi/180, 100, minLineLength=100,

maxLineGap=10) if lines is not None:

    for line in lines:

        x1, y1, x2, y2 = line # Corrected line: unpack 'line' directly

        cv2.line(image, (x1, y1), (x2, y2), (0,0,255), 2)

plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))

plt.title("Hough Line Detection")

plt.axis("off")

plt.show()
```

Output:

Result: the program is executed successfully.